

[BUG] Tensor Product of observables depends on order of operators and i

<https://github.com/PennyLaneAI/pennylane/issues/2263>

ROW 125

[BUG] Tensor Product of observables depends on order of operators and ignores wires information #2263

New issue

Closed



PatrickHuembeli opened on Mar 2, 2022 · edited by PatrickHuembeli

Edits ...

It seems that the tensor product operator `@` only depends on the order of the operators and not on their wire information. Not sure if this is intentional, but seems to be counterintuitive.

Expected behaviour

```
(qml.PauliZ(1)@qml.Identity(0)).matrix == (qml.PauliZ(0)@qml.Identity(1)).matrix is false
```

Or both `return` statements of the following circuit should return the same value

```
dev = qml.device('default.qubit.jax', wires=n_qubits//2)
np.random.seed(1)

@qml.qnode(dev, interface='jax', diff_method="backprop")
def qnode(params):
    for i in range(n_qubits//2):
        qml.RX(jnp.pi*np.random.rand(), wires=i)
    return qml.expval(qml.PauliZ(wires = 1)@qml.Identity(wires = 0))
    return qml.expval(qml.Identity(wires=0)@qml.PauliZ(wires=1))
```

Actual Behaviour:

```
(qml.PauliZ(1)@qml.Identity(0)).matrix == (qml.PauliZ(0)@qml.Identity(1)).matrix is true
but
(qml.Identity(0)@qml.PauliZ(1)).matrix == (qml.PauliZ(1)@qml.Identity(0)).matrix is false
```

And the circuit from above returns different values for the two different return statements.

System information

```
Name: PennyLane
Version: 0.21.0
Summary: PennyLane is a Python quantum machine learning library by Xanadu Inc.
Home-page: https://github.com/XanaduAI/pennylane
Author:
Author-email:
License: Apache License 2.0
Location: /Users/patrickhumbeli/anaconda3/envs/JAX/lib/python3.10/site-packages
Requires: appdirs, autograd, autoray, cachetools, networkx, numpy, pennylane-lightning, retworkx, scipy, semar
Required-by: PennyLane-Lightning
Platform info: macOS-12.1-x86_64-i386-64bit
Python version: 3.10.2
Numpy version: 1.21.5
Scipy version: 1.8.0
Installed devices:
- lightning.qubit (PennyLane-Lightning-0.21.0)
- default.gaussian (PennyLane-0.21.0)
- default.mixed (PennyLane-0.21.0)
- default.qubit (PennyLane-0.21.0)
- default.qubit.autograd (PennyLane-0.21.0)
- default.qubit.jax (PennyLane-0.21.0)
- default.qubit.tf (PennyLane-0.21.0)
- default.qubit.torch (PennyLane-0.21.0)
```

Existing GitHub issues

☒ I have searched existing GitHub issues to make sure the issue does not already exist.

Assignees

No one assigned

Labels

bug

Type

No type

Projects

No projects

Milestone

No milestone

Relationships

None yet

Development

No branches or pull requests

Notifications

Customize

Subscribe

You're not receiving notifications from this thread.

Participants



PatrickHuembeli added bug on Mar 2, 2022



antalszava on Mar 2, 2022

Contributor ...

Hi @PatrickHuembeli, thank you for reporting this! 😊

1.

```
(qml.PauliZ(1)@qml.Identity(0)).matrix == (qml.PauliZ(0)@qml.Identity(1)).matrix is false
```

This is intended because we don't make any assumptions on the wire order. The reason is that we may have custom wire labels, e.g.

```
(qml.PauliZ('a')@qml.Identity(0)).matrix == (qml.PauliZ(0)@qml.Identity('a')).matrix is false.
```

A new way of obtaining operator matrices using the `qml.matrix` top-level function has just made it into our `master` branch (see #2241). This function will then accept a `wire_order` argument that should help get the expected results. 😊

2.

As for the code snippet, thanks for reporting it, that should be working! We'll have a look into this.



josh146 on Mar 3, 2022

Member ...

Regarding point (2), I think the issue might be these lines here, inside the PennyLane `QubitDevice` class:

[pennylane/pennylane/_qubit_device.py](#)
Lines 782 to 783 in b494a2b

```
782     prob = self.probability(wires=observable.wires)
783     return self._dot(eigvals, prob)
```

Here, the eigenvalue order is being computed by *sorting the wires*:

```
>>> op1 = qml.PauliZ(1) @ qml.Identity(0)
>>> qml.eigvals(op1)
array([ 1, -1,  1, -1])
>>> np.kron([1, 1], [1, -1])
array([ 1, -1,  1, -1])
>>> op2 = qml.Identity(0) @ qml.PauliZ(1)
>>> qml.eigvals(op2)
array([ 1, -1,  1, -1])
```



So the eigenvalues of `op1` and `op2` are both being computed by assuming wire 0 < wire 1.

The problem is that the probability computation is not doing this; it is preserving the order of the observables in the tensor product, as it is being called with `op.wires`:

```
>>> op1.wires
<Wires = [1, 0]>
>>> op2.wires
<Wires = [0, 1]>
```



So there is a mismatch between how a tensor product observable computes its matrix, and how it reports its wires. The question then becomes: which is correct? As long as we ensure both have the same convention, this will be fixed.



josh146 on Mar 3, 2022

Member ...

If we look at the `Tensor` product itself, the eigenvalues are being sorted here:

[pennylane/pennylane/operation.py](#)
Lines 1845 to 1850 in f60407f

```
1845     # Sort observables lexicographically by the strings of the wire labels
1846     # TODO: check for edge cases of the sorting, e.g. Tensor(Hermitian(obs, wires=[0, 2]),
```

Goodnote

josh146 on Mar 3, 2022

Member ...

If we look at the `Tensor` product itself, the eigenvalues are being sorted here:

[pennylane/pennylane/operation.py](#)
Lines 1845 to 1850 in [f60407f](#)

```
1845 # Sort observables lexicographically by the strings of the wire labels
1846 # TODO: check for edge cases of the sorting, e.g. Tensor(Hermitian(obs, wires=[0, 2]),
1847 # Hermitian(obs, wires=[1, 3, 4]))
1848 # Sorting the observables based on wires, so that the order of
1849 # the eigenvalues is correct
1850 obs_sorted = sorted(self.obs, key=lambda x: [str(l) for l in x.wires.labels])
```

If we remove this sorting, this fixes this bug!



Jaybsoni mentioned this on Mar 3, 2022

[expval](#) ignores wire labels and uses position implicitly for tensor product ops #2276

Jaybsoni on Mar 3, 2022

Contributor ...

Hey @josh146,

That is exactly correct!

I think if we remove the sort in the eigenvalues then we will be taking the convention in which the wire labels are ignored. While this makes sense in the case where custom wire labels are used (i.e wires = ["a", "b", "c"]), it is less intuitive than using the sorted order as the wires provided are ignored. I think in the majority of cases, taking the sorted convention makes more sense/.

Regarding point (2), I think the issue might be these lines here, inside the PennyLane `QubitDevice` class:

[pennylane/pennylane/_qubit_device.py](#)
Lines 782 to 783 in [b494a2b](#)

```
782 prob = self.probability(wires=observable.wires)
783 return self._dot(eigvals, prob)
```

Here, the eigenvalue order is being computed by *sorting the wires*:

```
>>> op1 = qml.PauliZ(1) @ qml.Identity(0)
>>> qml.eigvals(op1)
array([ 1, -1,  1, -1])
>>> np.kron([1, 1], [1, -1])
array([ 1, -1,  1, -1])
>>> op2 = qml.Identity(0) @ qml.PauliZ(1)
>>> qml.eigvals(op2)
array([ 1, -1,  1, -1])
```



So the eigenvalues of `op1` and `op2` are both being computed by assuming wire 0 < wire 1.

The problem is that the probability computation is not doing this; it is preserving the order of the observables in the tensor product, as it is being called with `op.wires`:

```
>>> op1.wires
<Wires = [1, 0]>
>>> op2.wires
<Wires = [0, 1]>
```



So there is a mismatch between how a tensor product observable computes its matrix, and how it reports its wires. The question then becomes: which is correct? As long as we ensure both have the same convention, this will be fixed.



josh146 on Mar 4, 2022

Member ...

I think if we remove the sort in the eigenvalues then we will be taking the convention in which the wire labels are ignored. While this makes sense in the case where custom wire labels are used, it is less intuitive than using the sorted order as the wires provided are ignored. I think in the majority of cases, taking the sorted convention makes more sense/.

@Jaybsoni something to keep in mind is that we shouldn't treat integer wires differently from other wire labels, as this introduces more inconsistency!

Say we consider the case

```
op1 = qml.PauliZ(1) @ qml.Identity(0)
op2 = qml.PauliZ(0) @ qml.Identity(1)
op3 = qml.PauliZ("a") @ qml.Identity(7)
```



Since we have no context as to what the canonical wire ordering *is*, I would argue that it doesn't make sense to attempt to order the eigenvalues by default! Instead, by default, we should expect all three of the above to return eigenvalues in the same order:

```
>>> np.kron([1, -1], [1, 1])
array([ 1,  1, -1, -1])
```



However, this is not the case, since `Tensor.eigvals` is doing an implicit sort by default:

```
>>>> qml.eigvals(op3)
array([ 1, -1,  1, -1])
>>> qml.eigvals(op2)
array([ 1,  1, -1, -1])
>>> qml.eigvals(op1)
array([ 1, -1,  1, -1])
```



It is making an assumption that may have no basis.

Instead, we should do the following here:

- If no canonical wire order is known, `qml.eigvals()` should make no assumptions, and simply use the ordering of the tensor product observables.
- If a canonical wire ordering *is* known, we should be able to pass it to `qml.eigvals()` to take into account.

This is identical to how `qml.matrix()` behaves, and we should bring `qml.eigvals()` inline with this behaviour:

```
>>> np.all(qml.matrix(op1) == qml.matrix(op2))
True
>>> np.all(qml.matrix(op1) == qml.matrix(op3))
True
>>> np.all(qml.matrix(op1, wire_order=[0, 1]) == qml.matrix(op2, wire_order=[0, 1]))
False
```



josh146 on Mar 4, 2022

Member ...

So this is partly why I lean towards *not* sorting eigvals by default, while leaving the permutation in `probs` --- it seems a lot cleaner and consistent



antalszava mentioned this on Mar 9, 2022

🔗 [Remove sorting the underlying probability vector for qml.expval #2292](#)

antalszava on Mar 9, 2022

Contributor ...

Hi @josh146,

antalszava on Mar 9, 2022

Contributor ...

Hi @josh146,

The problem is that the probability computation is not doing this; it is preserving the order of the observables in the tensor product

Isn't this being done here?

[pennylane/pennylane/_qubit_device.py](#)
Line 757 in 67205b6

```
757 basis_states = basis_states[:, np.argsort(np.argsort(device_wires))]
```

This might even be the cause of the issue and avoiding this reordering might be a start on a viable solution. If no reordering is done, then the amplitudes remain in a big-endian ordering. It should then suffice to simply just to sort the observables (something that's already done). The reason why this sorting will be important is that as per the original example in 2., a user might provide the same observable in different forms and we'd like the same expectation to be calculated. Since the order of the tensor product terms (and hence the eigenvalues) is what may change in the user code, logically, it would make sense to change that only.

Attempting an even more complex example like `qml.Identity(wires=0) @ qml.Identity(wires=1) @ qml.PauliZ(wires=2)`

#2292 attempts this approach (having started first with changing the eigvals logic) with a few caveats:

- Tests that use `qml.Hermitian` fail: these cases depended on the reordering of amplitudes;
- Omitting the reordering in `marginal_prob` would likely require the intro of some new option or API: the `qml.probs` pipeline would still require this option to be there, whereas `qml.expval` may not.



josh146 on Mar 9, 2022 · edited by josh146

Edits Member ...

@antalszava I see your point 🙄 My worry is that sorting of eigenvalues is a lot more difficult, both conceptually and logically, compared to sorting of probabilities.

E.g., by using the following diff, consider the output which solves this bug with a diff of `+4/-8` :

▶ Click to view PennyLane diff

```
dev = qml.device("default.qubit", wires=2)
x = np.array(0.543, requires_grad=True)
```

```
op1 = qml.PauliZ(1) @ qml.Identity(0)
op2 = qml.Identity(0) @ qml.PauliZ(1)
```

```
@qml.qnode(dev)
def circuit(x, op):
    qml.RX(x, wires=0)
    qml.RY(x, wires=1)
    qml.CNOT(wires=[0, 1])
    return qml.expval(op)
```

```
>>> print(circuit(x, op1))
Probability device wires: [1, 0]
eigvals: [ 1.  1. -1. -1.]
probs: [0.86133473 0.00517231 0.06674648 0.06674648]
0.7330140825988753
```

```
>>> print(circuit(x, op2))
Probability device wires: [0, 1]
eigvals: [ 1. -1.  1. -1.]
probs: [0.86133473 0.06674648 0.00517231 0.06674648]
0.7330140825988753
```

```
>>> print(circuit(x, qml.PauliZ(1)))
Probability device wires: [1]
eigvals: [ 1. -1.]
probs: [0.86650704 0.13349296]
0.7330140825988753
```

Having said that, your point is very valid --- it is fair to either remove the eigvals sorting, or remove the probs sorting, both would solve this issue.

z

```
>>> print(circuit(x, qml.PauliZ(1)))
Probability device wires: [1]
eigvals: [ 1. -1.]
probs: [0.86650704 0.13349296]
0.7330140825988753
```

Having said that, your point is very valid --- it is fair to either remove the eigvals sorting, or remove the probs sorting, both would solve this issue.

Note: If we do keep the `eigvals` sorting, we should require passing a `wire_order` argument,

```
>>> op.get_eigvals(wire_order=...)
```

If `wire_order=None`, then no eigenvalue sorting is performed. This ensures that:

- `eigvals` behaves identically to other representations, like matrices
- we no longer treat integer wires differently to other wire labels.

Once this is done, we then have a choice inside the simulator:

1. Choice one: eigenvalue sorting, no probs sorting

```
eigvals = self._asarray(observable.get_eigvals(wire_order=self.wires), dtype=self.R_DTYPE)
prob = self.probability()
```

2. Choice two: no eigenvalue sorting, probability sorting

```
eigvals = self._asarray(observable.get_eigvals(), dtype=self.R_DTYPE)
prob = self.probability(observable.wires)
```

My intuition is that (2) will be easier to implement than (1).



[antalszava](#) mentioned this on Mar 9, 2022

[\[Prototype\] Probs sort no eigval sort #2293](#)



antalszava on Mar 9, 2022

Contributor ...

Hi [@josh146](#),

My worry is that sorting of eigenvalues is a lot more difficult, both conceptually and logically, compared to sorting of probabilities.

Agreed! However, it seems this would be the way that we eventually get the correct results in more intricate cases too.

While removing the observable sorting does help with the original 2-term tensor product case, it [fails when we have 3-terms in the TP](#). See the `test_pauliz_identity` device test and the [#2293](#) prototype.

The same test passes on [#2292](#).



Jaybsoni on Mar 22, 2022

Contributor ...

This issue has been resolved in this [PR](#)



Jaybsoni closed this as [completed](#) on Mar 22, 2022

expval ignores wire labels and uses position implicitly for tensor product ops

#2276

<> Code

Merged Jaybsoni merged 87 commits into master from wire_order_bug on Mar 18, 2022

Conversation 63 Commits 87 Checks 0 Files changed 6

+327 -11



Jaybsoni commented on Mar 3, 2022 · edited

Contributor

Context:
Consider the following circuit:

```
dev = qml.device("default.qubit", wires=3)

@qml.qnode(dev)
def circ():
    qml.Hadamard(wires=0)
    qml.CNOT(wires=[0, 2])
    return qml.expval(qml.Identity(wires=0) @ qml.PauliZ(wires=1)) # --> 0.9999999999999998
    # return qml.expval(qml.Identity(wires=1) @ qml.PauliZ(wires=0)) # --> 0.9999999999999998
    # return qml.expval(qml.PauliZ(wires=0) @ qml.Identity(wires=1)) # --> 0.0
    # return qml.expval(qml.PauliZ(wires=1) @ qml.Identity(wires=0)) # --> 0.0
```

But the expected behaviour should be:

```
...
return qml.expval(qml.Identity(wires=0) @ qml.PauliZ(wires=1)) # --> 0.9999999999999998
# return qml.expval(qml.Identity(wires=1) @ qml.PauliZ(wires=0)) # --> 0.0
# return qml.expval(qml.PauliZ(wires=0) @ qml.Identity(wires=1)) # --> 0.0
# return qml.expval(qml.PauliZ(wires=1) @ qml.Identity(wires=0)) # --> 0.9999999999999998
```

It seems as though the wire labels are being ignored and instead the position of the operations in the tensor product are being used to determine how to compute the expectation value.

Additionally:

This PR also addresses the following bug which was discovered during the testing of this feature:

```
dev = qml.device("default.qubit", wires=3)
theta = 0.432
phi = 0.123
varphi = -0.543

@qml.qnode(dev)
def circuit(o):
    qfunc()
    return qml.expval(qml.apply(o))

obs0 = qml.PauliZ(wires=2)

obs1 = qml.Identity(wires=0) @ qml.Identity(wires=1) @ qml.PauliZ(wires=2)
obs2 = qml.Identity(wires=1) @ qml.Identity(wires=0) @ qml.PauliZ(wires=2)

obs3 = qml.Identity(wires=0) @ qml.PauliZ(wires=2) @ qml.Identity(wires=1)
obs4 = qml.Identity(wires=1) @ qml.PauliZ(wires=2) @ qml.Identity(wires=0)

obs5 = qml.PauliZ(wires=2) @ qml.Identity(wires=0) @ qml.Identity(wires=1)
obs6 = qml.PauliZ(wires=2) @ qml.Identity(wires=1) @ qml.Identity(wires=0)

obs_lst = [obs0, obs1, obs2, obs3, obs4, obs5, obs6]

for i, o in enumerate(obs_lst):
    print(f"----- {i} ----- \n{circuit(o)}")
```

Which produces the following result:

```
----- 0 -----
0.8561624160163042
----- 1 -----
0.8561624160163042
----- 2 -----
0.8561624160163042
----- 3 -----
```

Reviewers

josh146

antalszava

Assignees

No one assigned

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Successfully merging this pull request may close these issues.

None yet

Notifications

Customize

Subscribe

You're not receiving notifications from this thread.

9 participants



Zotero

```
for i, o in enumerate(obs_lst):
    print(f"----- {i} ----- \n{circuit(o)}")
```

Which produces the following result:

```
----- 0 -----
0.8561624160163042
----- 1 -----
0.8561624160163042
----- 2 -----
0.8561624160163042
----- 3 -----
0.8561624160163042
----- 4 -----
0.9081301906949608
----- 5 -----
0.9924450321351934
----- 6 -----
0.8561624160163042
```

In the above result we notice that the expectation values computed for observables 4, 5 do not match the others.

Description of the Change:

The `expval(self, observable, shot_range=None, bin_size=None)` function defined in `_qubit_device.py` uses the eigenvalues of the provided tensor product observable and the probability vector of the state to compute the expectation value when `shots=None`.

The first bug arises because of lexicographic sorting of the eigvals based on the wires the observables act on in the function `qml.Tensor.get_eigvals()`. This results in the eigvals of the two observables `PauliZ @ Identity` and `Identity @ PauliZ` being the same.

This is addressed by removing the sorting in the `get_eigvals()` function.

The second bug arises because of the way we permute the probability vector before computing the expectation value. The wire order of the observables is passed into the `device.probability()` function which then computes the marginal probability distribution over the observable wires and permutes the distribution according to the wire order of the observable wires. This permutation is incorrect and instead we should be using the inverse of this permutation. The reason such a bug was not caught sooner is because in most cases the tensor product observables usually contain 2 terms at most and/or there is only one qubit flipped out of position. In these common cases, the wire order permutation is its own inverse, and thus produces the correct expected value. But in situations with more than one qubit flip, it breaks.

This is addressed by computing the permutation before hand, then permuting the wires before they are passed into the probability function.

Related GitHub Issues:

This PR closes this [issue](#)



Jaybsoni added 2 commits 3 years ago

- sorted wires to match eig vals when computing expectation value! 55ae816
- Merge branch 'master' into wire_order_bug Verified f474cb7



github-actions (bot) commented on Mar 3, 2022

Contributor ...

Hello. You may have forgotten to update the changelog!
Please edit [doc/releases/changelog-dev.md](#) with:

- A one-to-two sentence description of the change. You may include a small working example for new features.
- A link back to this PR.
- Your name (or GitHub username) in the contributors section.

